

149. `bande()` devient une fonction de la console : `#include <bande>`

La seconde sorte d'`#include`, avec des chevrons : `bande()` a été ajoutée à la console elle-même. On ne l'écrit plus, on la demande.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : `#include <bande>`, avec des **chevrons**. `bande()` n'est plus dans `outils.cpp` (il n'y reste que `pile()`) : elle fait maintenant partie **de la console**, comme `poser()` ou `texte()`. On ne l'écrit plus, on la **demande**.

Guillemets ou chevrons, la différence en une phrase : `#include "outils.cpp"` verse **ton fichier** ; `#include <bande>` demande **une fonction de la console**, que le compilateur connaît déjà et n'ajoute à la cartouche que si le programme l'appelle.

Sans la ligne, le compilateur refuse : efface `#include <bande>` et lance. Le message dit : « il faut `#include <bande>` pour employer `bande()` ». C'est la règle de toute la console : ce qu'on emploie, on l'inclut par son nom.

Comment `bande()` est entrée dans la console. Il a fallu toucher **trois fichiers du projet** (pas le programme : le compilateur lui-même). On les ouvre dans un éditeur de texte, à côté d'`index.html` :

1. `compilateur/emetteur.js` — **le code de la fonction**. On y écrit sa source, **en C, exactement comme dans `outils.cpp`**, dans une constante, `SOURCE_BANDE`, entre deux accents graves (la touche AltGr + 7) : le texte de la fonction, mot pour mot. Puis on l'inscrit dans le tableau `FONCTIONS_EN_C`, juste en dessous : `bande: SOURCE_BANDE`,. Le compilateur ajoute ce texte au programme **seulement** s'il appelle `bande()`.

2. `compilateur/inclusion.js` — **le nom à inclure**. Dans le tableau `BIBLIOTHEQUES`, une ligne : `bande: 'pose la même tuile plusieurs fois, de gauche à droite',`. C'est elle qui rend `#include <bande>` valable, et la phrase sert de commentaire quand l'atelier écrit les `#include` tout seul.

3. `aide-fonctions.js` — **l'aide de l'éditeur**. Dans `FONCTIONS` : `bande: { args: ['colonne', 'ligne', 'tuile', 'longueur'], dit: '...' },`. L'éditeur propose alors `bande` quand on tape « ban... », et montre les arguments pendant qu'on les écrit.

Un détail : dans la console, `bande()` n'écrit pas `#include <poser>`. Ce que la console ajoute elle-même n'a rien à inclure : le compilateur le sait (c'est le drapeau `deLaConsole`).

Après avoir changé ces fichiers : recharge la page avec **Ctrl+F5**. Le compilateur est relu, et `#include <bande>` marche aussitôt.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <bande>` → [148.1. La fonction `bande\(\)` — la même tuile, plusieurs fois](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1010 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
bande()         38 o ← main() – une fonction fournie par le langage
pile()          42 o ← main() – ta fonction
main()          83 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AdresseCase     10 o ← poser() – trouve l'adresse d'une case de l'écran
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 2      38 o sur place, dans bande(), pile() + AdresseCase (10 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
bande() × 3      57 o sur place, dans main()
pile() × 1       19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le changement : bande() vient de la CONSOLE, plus d'utils.cpp.
//
// #include <bande>      des CHEVRONS : une fonction de la console
// #include "utils.cpp"  des GUILLEMETS : un fichier à moi

#include <ALPHABET>
#include <bande>         // NOUVEAU : pose la même tuile plusieurs fois, de gauche à droite
#include "utils.cpp" // il ne contient plus que pile()
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    bande(2, 5, ALPHABET[0], 10); // la bande() de la console
    bande(2, 7, ALPHABET[1], 6);
    bande(2, 9, ALPHABET[2], 3);
    pile(15, 4, ALPHABET[3], 6); // la pile() d'utils.cpp

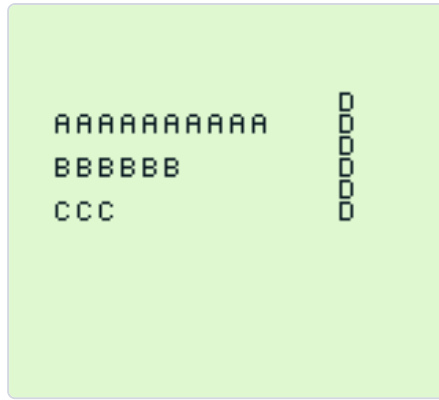
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le même écran : trois bandes et une pile. `bande()` vient maintenant de la console.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la `bande()` de la console pose dix A
- ✓ six B, trois C
- ✓ et la `pile()` d'`outils.cpp` est toujours là

Ensuite : **150. Ta fonction passe avant celle de la console** — chapitre 25 : Tes propres `#include` : ajouter une fonction. ·
`gameboy3` — écrire une cartouche Game Boy en C++